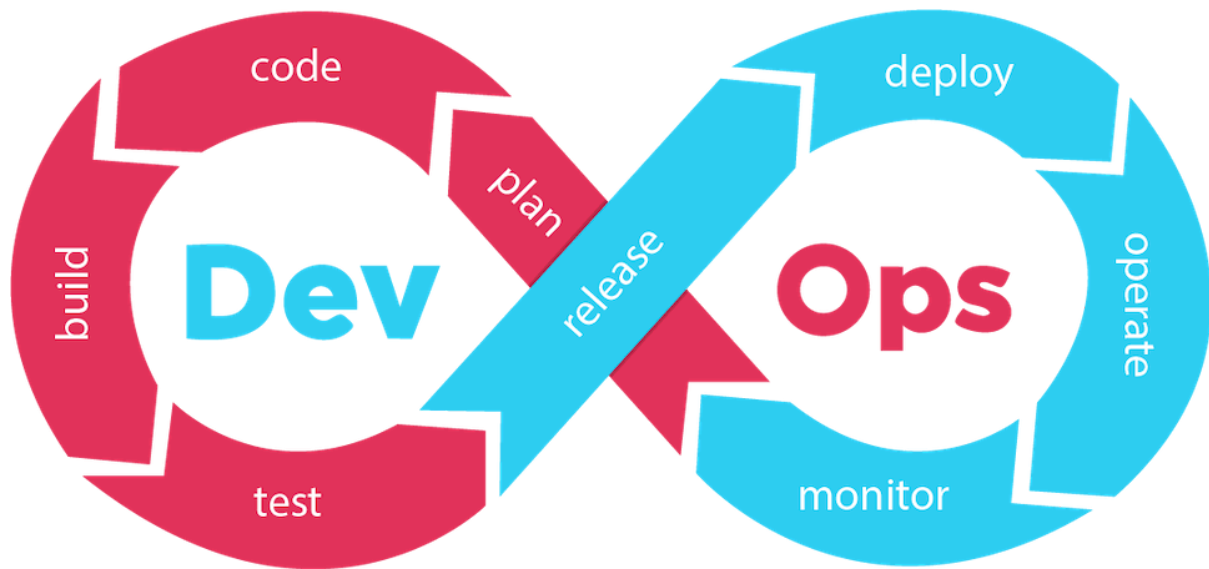




Interview Question and Answers

سوالات مصاحبه Devops قسمت ششم



این قسمت containerization
(DOCKERFILE :ENTRYPOINT VS CMD)



سوال مصاحبه « در DOCKERFILE فرق CMD و ENTRYPOINT چیه؟ »

اول: چند مدل میشه این سوال رو پرسید؟ 🧠

♦ مدل 1: مستقیم و ساده

• "در DOCKERFILE فرق CMD و ENTRYPOINT چیه؟"

♦ مدل 2: عملی

• "اگر هر دو دستور در DOCKERFILE باشن، کدوم اجرا میشه؟"

♦ مدل 3: چالش

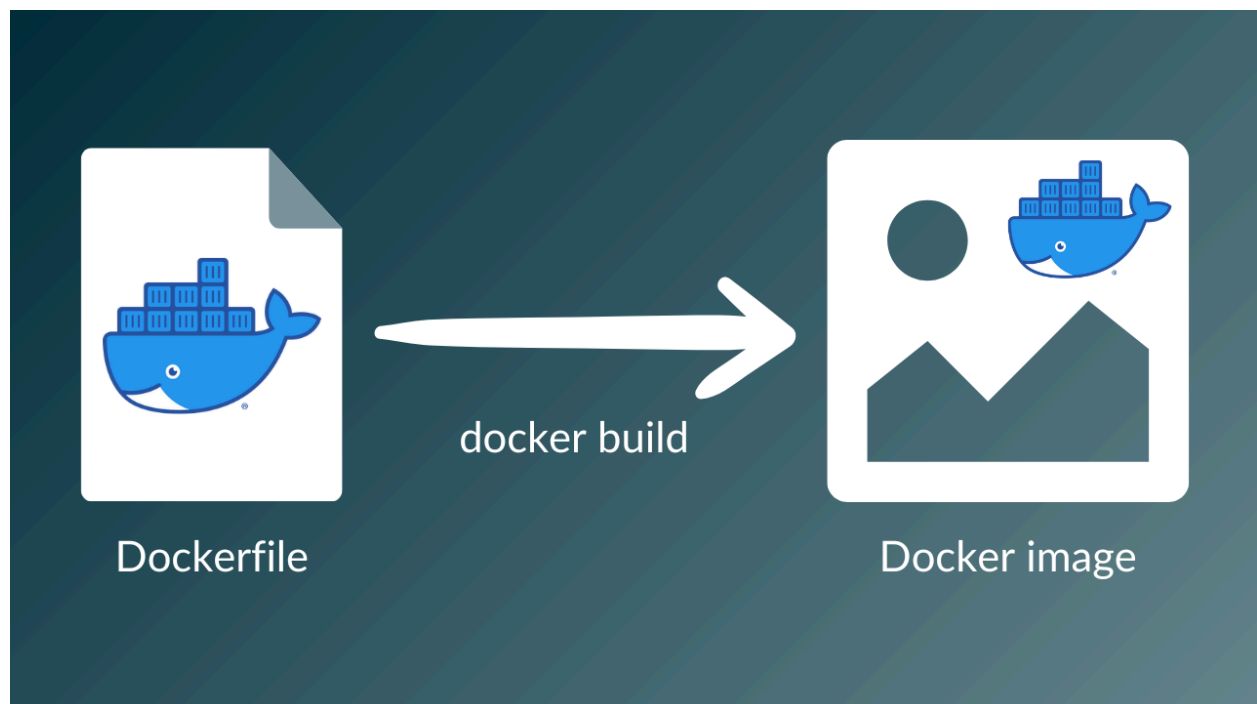
• "چطور میشه باعث شد که همیشه یک برنامه اجرا بشه و قابل override نباشه؟"

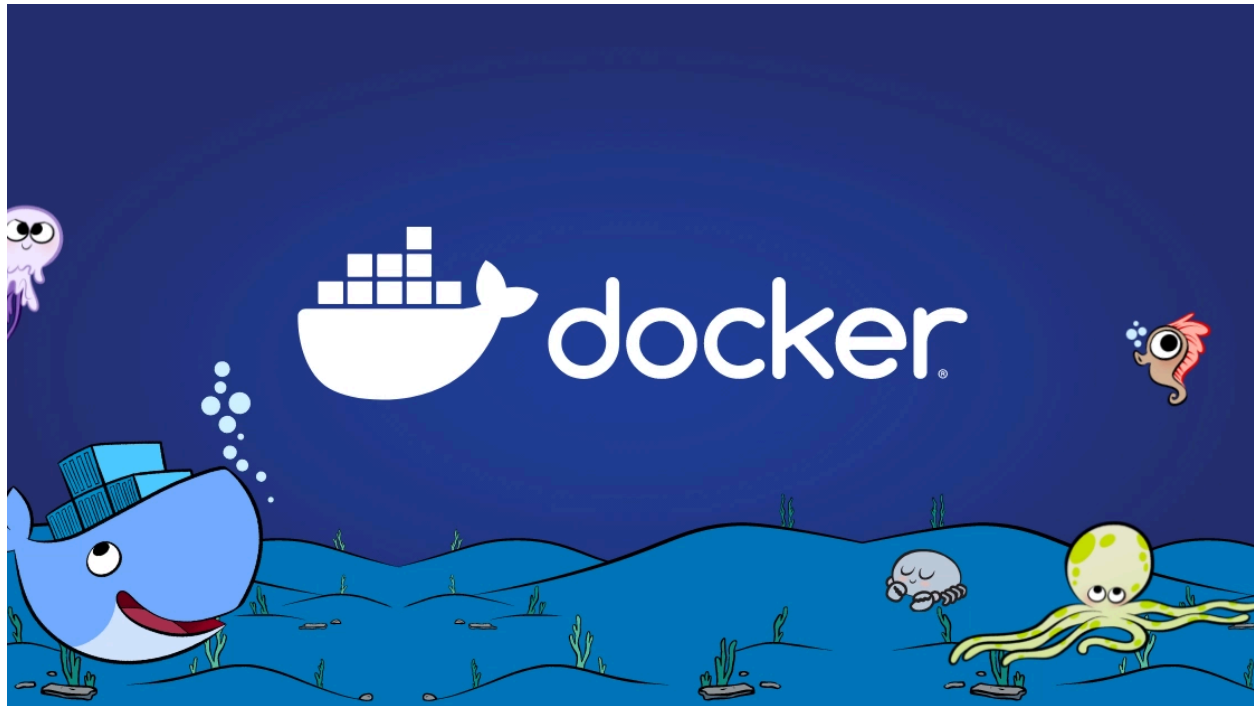
♦ مدل 4: Kubernetes scenario

• "چرا در کانتینرهایی که روی k8s ران می‌شن، ENTRYPOINT بهتر از CMD است؟"

♦ مدل 5: Debug scenario

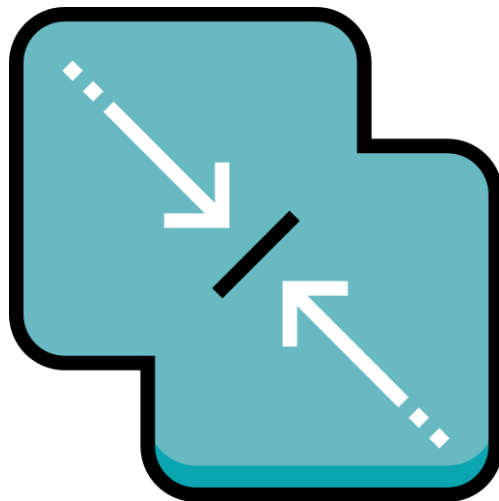
• "اگر بخوای برنامه اصلی رو موقتاً عوض کنی، کدوم رو تغییر میدی؟"





حالا بریم سراغ جواب ساده و روون:

در Dockerfile از **ENTRYPOINT** برای تعریف برنامه اصلی استفاده می‌کنیم که همیشه باید اجرا شود، و از **CMD** برای تعیین آرگومان یا دستور پیش‌فرض که کاربر می‌تواند آن را تغییر دهد.



2) رفتار ترکیبی آنها

اگر هر دو در داکر فایل باشند:

```
ENTRYPOINT ["python3"]  
CMD ["app.py"]
```

اجرای container به صورت ذیل :

```
docker run myimage
```

➔ معادل اجرای دستور ذیل خواهد بود :

```
python3 app.py
```

اما اگر موقع ران اجرا کنی:

```
docker run myimage test.py
```

➔ معادل اجرای دستور ذیل خواهد بود :

```
python3 test.py
```



اگر هر دو باشند، **CMD** به عنوان آرگومان به **ENTRYPOINT** پاس داده می‌شود.

3 تفاوت override شدن

تفاوت مهم این است که **CMD** با اجرای **docker run IMAGE** **something** کاملاً جایگزین می‌شود، ولی **ENTRYPOINT** فقط آرگومان‌هایش تغییر می‌کند مگر اینکه با **entrypoint--** **override** شود.

اگر در **DOCKERFILE** بنویسیم

```
CMD ["echo", "Hello"]
```

آنگاه اگر به صورت ذیل اجرا کنیم

```
docker run myimage Hi
```

خروجی:

Hi

- چون **CMD** کامل **override** شد.

حالا اگر تو داکر فایل بنویسیم

```
ENTRYPOINT ["echo", "Hello"]
```

واینجوری اجراش کنیم

```
docker run myimage Hi
```

خروجی:

Hello Hi

- چون فقط آرگومان‌ها اضافه شدند.



در محیط‌هایی مثل Kubernetes، بهتر است از **ENTRYPOINT** با فرمت **exec** استفاده کنیم تا سیگنال‌ها درست به **process** اصلی برسند.

kubernetes

برای اینکه اهمیت این موضوع رو متوجه بشیم باید موضوع زیر رو متوجه بشیم .

وقتی یک کانتینر اجرا می‌کنی، **process اصلی** که توش اجرا میشه باید مستقیماً همون برنامه‌ای باشه که میخوای اجرا بشه.
اگر این برنامه **زیرمجموعه یک شل (bin/sh/)** باشه، سیگنال‌هایی مثل **SIGTERM** و **SIGINT** اول میرن به شل، نه به برنامه اصلی.
این مشکل می‌تونه باعث بشه:

- وقتی کانتینر رو stop می‌کنی، برنامه به موقع بسته نشه.
- در Kubernetes، کانتینر نتونه به موقع shutdown (graceful shutdown) شه.

✓ exec form (فرمت ترجیحی)

```
ENTRYPOINT ["python3", "app.py"]
```

اگر در داکر فایل اینجوری بنویسیم

- اینجا Docker مستقیماً **python3** رو به عنوان **process اصلی (PID 1)** اجرا می‌کنه.
- هیچ شلی وسط ماجرا نیست.
- سیگنال‌ها مستقیماً به **python3** می‌رسه.
- در Kubernetes یا Docker stop → برنامه درست و سریع بسته میشه.

⚠ shell form

```
ENTRYPOINT python3 app.py
```

- اینجا Docker اول یک شل **(bin/sh -c/)** اجرا می‌کنه و بعد دستور رو توش می‌فرسته.
- **process اصلی** در واقع **bin/sh/** هست، نه **python3**.
- سیگنال‌ها اول میرن به شل، و اگر شل forward نکنه، برنامه ممکنه دیر یا اصلاً بسته نشه.
- در Kubernetes این می‌تونه باعث بشه Pod تو وضعیت **Terminating** گیر کنه.

exec form

```
PID 1: python3 app.py
```

shell form

```
PID 1: /bin/sh -c "python3 app.py"  
└─ PID 7: python3 app.py
```

در حالت shell form، برنامه شما دیگه PID 1 نیست و سیگنال‌های توقف مستقیماً بهش نمی‌رسه.

چرا Kubernetes حساسه؟

K8s وقتی یک کانتینر رو می‌خواد متوقف کنه، به process اصلی (PID 1) سیگنال می‌فرسته. اگر PID 1 شل باشه و شل سیگنال رو پاس نده، کانتینر مجبور میشه بعد از timeout با SIGKILL کشته بشه → graceful shutdown انجام نمیشه → دیتای نیمه‌کاره، corruption و ...

خلاصه نکته مصاحبه‌ای

همیشه برای برنامه اصلی از exec form استفاده کن (ENTRYPOINT)
["executable", "arg1",
["arg2"] تا process اصلی مستقیماً همون
برنامه باشه و سیگنال‌ها درست مدیریت بشن.
shell form رو فقط وقتی استفاده کن که
واقعاً لازم داری شل ویژگی‌هایی مثل
wildcard (*) یا pipe (|) رو پردازش کنه.



✓ جمع بندی

ویژگی	ENTRYPOINT	CMD
هدف اصلی	مشخص کردن برنامه یا اسکریپت اصلی که کانتینر باید اجرا کند.	مشخص کردن آرگومان‌ها یا دستور پیش‌فرض برای اجرای برنامه.
قابل override شدن	به‌طور پیش‌فرض با اجرای <code>docker run IMAGE ...</code> نمی‌توان کل دستور را تغییر داد، فقط آرگومان‌ها را می‌توان اضافه کرد (مگر با <code>--entrypoint</code>).	با اجرای <code>docker run IMAGE</code> ... به‌طور کامل قابل override شدن است.
استفاده معمول	وقتی می‌خواهی کانتینر همیشه یک برنامه مشخص را اجرا کند.	وقتی می‌خواهی مقدار پیش‌فرض یا آرگومان پیش‌فرض بدهی که قابل تغییر باشد.
تعداد در Dockerfile	فقط یک بار قابل تعریف است، آخرین تعریف اعمال می‌شود.	فقط یک بار قابل تعریف است، آخرین تعریف اعمال می‌شود.